

Säkerhetsgranskning: VoiceStudio

Repo github.com/debpalash/VoiceStudio · commit eaf8bb9 · licens AGPL-3.0 · granskad 13 september 2026 · 2 240 spårade filer

Slutsats: ingen skadlig kod hittad

Jag hittade inga bakdörrar, inga dolda installationshakar, ingen obfuserad kod, ingen inhämtning av lösenord/nycklar och ingen oannonserad utgående trafik. Tvärtom är projektet ovanligt säkerhetsmedvetet för att vara ett hobby-nära open source-projekt: det har egen SSRF- och DNS-rebinding-testsvit, gitleaks som blockerar merge, CodeQL, bandit och beroendegranskning i CI, samt en genomtänkt behörighetsmodell. Kvarstående risker är strukturella (osignerade Windows-installerare, modellfiler från Hugging Face, `curl | sh`-installation) — inte tecken på uppsåt.

Kort analogi

Tänk på det som en husbesiktning. Jag har gått igenom grunden (finns det något inmurat som inte ska vara där?), elcentralen (vad kopplas ut från huset?), ytterdörrarna (vem kan komma in?) och besiktningsprotokollen från tidigare år (vad har andra redan hittat?). Huset är välbyggt och tidigare anmärkningar är åtgärdade. Det som återstår är att ytterdörren levereras utan certifierat lås i Windows-versionen, och att du själv bär in möbler (AI-modeller) från ett lager som du inte äger.

1. Vad jag granskat

Område	Vad jag letade efter	Resultat
Installationshakar	<code>preinstall/postinstall/prepare</code> i samtliga <code>package.json</code> — klassiskt npm-kapningsmönster	Ren — noll förekomster
Droppers	<code>curl ... bash</code> mot okända värdar, <code>base64 -d</code> vidarekopplat till skal, kodavkodning + <code>exec</code>	Ren — alla träffar är dokumenterade installerare mot egna/kända domäner
Kodexekvering	<code>eval()</code> , <code>exec()</code> , <code>os.system()</code> , <code>shell=True</code> , <code>pickle.loads</code> , osäker <code>yaml.load</code>	Ren — inga träffar i backend eller skript
Nyckelstöld	<code>~/.ssh</code> , <code>.aws/credentials</code> , <code>.netrc</code> , <code>keychain</code> , webbläsarkakor, plånboksfiler	Ren — inga träffar
Utgående trafik	Samtliga domäner i körbar kod (<code>py/js/ts/rs/sh/ps1</code>) listade och klassade	Ren — endast Hugging Face, PyTorch, GitHub, egna domäner och opt-in-telemetry
Committade binärer	ELF/Mach-O/PE-filer i git-historiken (klassiskt gömställe)	Ren — inga alls
CI-kedjan	<code>pull_request_target/workflow_run</code> med utcheckning av PR-kod ("pwn request") och brett exponerade secrets	Ren — mönstret används inte; secrets bara i <code>release/docker/evals</code>
Nätverk	Standardbindning, CORS, autentisering på API:et	Ren — loopback som standard, PIN/API-nyckel för LAN

Område	Vad jag letade efter	Resultat
Deserialisering	<code>torch.load()</code> utan <code>weights_only</code> (pickle = kodexekvering)	Medel — 2 träffar, båda i utvärderingsverktyg

2. Vad som faktiskt lämnar din dator

Påståendet i inlägget du såg — ”ingen audio lämnar någonsin din dator” — stämmer, och det är hårdare genomdrivet än marknadsföringstexten antyder. Det finns telemetri, men den är **avstängd som standard** och konstruerad så att den inte kan läcka innehåll:

- **Två oberoende grindar** måste båda vara sanna: en konfigurerad mottagare och ditt uttryckliga val. Standardvärdet i koden är `False`.
- **Hård avstängning** med miljövariabeln `OMNIVOICE_ANALYTICS_DISABLED=1`, som slår ut båda grindarna.
- **Allowlist i koden**, inte i en policytext. Endast ~15 fält får passera (motor-id, språk, sekunder, antal tecken — aldrig texten, aldrig filnamn, aldrig röstnamn). Ett fält som inte står på listan kastas.
- **Autocapture och session recording är explicit avstängda** — SDK:ns standardläge skulle annars skicka texten du klickar på, vilket i den här appen är just manuset du ska syntetisera.
- **Stacktraces skickas medvetet inte**, eftersom de kan bära absoluta sökvägar och Hugging Face-tokens. Bara undantagets klassnamn.
- **Identitet** är ett slumpat UUID per installation — inte härlett från hårdvara, värddamn eller användarnamn.

Mottagaren är PostHog på EU-värd (`eu.i.posthog.com`). Projekt-token ligger committad i klartext — det är avsiktligt och ofarligt: det är en publishable skrivnyckel som bara pekar ut en destination och inte kan läsa data. Avinstallationsskriptet skickar en sista `app_uninstalled`-signal, men endast om du var opt-in — annars skickas ingenting och inget skrivs ut.

3. Vad andra redan har hittat

Det finns **inga publicerade säkerhetsråd (GitHub Security Advisories)** och **inga CVE:er** för projektet, och jag hittade inga användarrapporter om skadlig kod eller dataläckage. Däremot finns riktiga säkerhetsfynd som redan är åtgärdade — vilket är ett gott tecken, eftersom det visar att någon faktiskt tittar:

Fynd	Vad det innebar	Status
Admin-rutter nåbara utan autentisering (#1213)	I serverläge kunde en klient på ett betrott nätverk nå <code>/system/*</code> och <code>/api/settings/*</code> utan nyckel. Projektet klassar rutterna själv som ”RCE-class” — alltså kodexekvering. Den korta delnings-PIN:en gav också admin.	Fixat i v0.4.0. Kräver nyckel eller äkta loopback.
35 + 5 beroendeaviseringar (#1456, #2030, #2031)	Sårbarheter i Python- och Rust-beroenden, samt <code>protobuf</code> / <code>transformers</code> i <code>CosyVoice</code> 3.	Fixat genom uppgräderingar

Fynd	Vad det innebar	Status
Ogiltig röstprofil kunde sparas (#1141)	Fritext i ett profilfält kunde spara en profil som kraschade varje framtida generering — "återutnyttjat tre gånger via olika klienter".	Fixat — servern sanerar nu alla profiltyper
Osignerade Windows-installerare (#1712)	Authenticode-signering av stabila Windows-installerare är avförd som "not planned". SmartScreen kommer varna, och du kan inte kryptografiskt verifiera vem som byggt filen.	Ej planerat — öppen, medveten avvägning
Saknad vattenstämpel i API (#1169)	/v1/audio/speech returnerade omärkt ljud — regelefterlevnad (EU AI Act art. 50.2), inte en attackvektor.	Fixat

4. Kvarstående risker — rangordnade

#	Risk	Varför den finns	Vad du gör åt den
1	Modellfiler från tredje part (störst)	Appen laddar ner AI-modeller från Hugging Face. En .pt/.bin-fil är pickle-serialiserad Python och kan köra kod när den läses in. Det är hela ekosystemets problem, inte det här projektets.	Håll dig till de motorer appen föreslår. Projektets egen säkerhetspolicy säger uttryckligen: aldrig privat distribuerade modellarkiv, aldrig körbara filer som följer med ett modellarkiv.
2	Osignerade installerare (Windows + macOS)	Windows saknar Authenticode-signatur. macOS-DMG:n är ad-hoc-signerad men inte notariserad — Gatekeeper blockerar första starten. Du kan inte verifiera utgivaren på någon av dem, bara checksumman.	Verifiera SHA256 mot checksums-filen i GitHub-releasen på båda plattformarna, eller bygg från källkod. På macOS: högerklicka → Öppna vid första starten.
3	curl ... sh-installation	Installationsskriptet gör SHA256-kontroll, men hoppar över den om inget shasum/sha256sum finns på maskinen — och checksum-filen kommer från samma källa som binären.	Ladda ner skriptet först, läs det, kör det sedan. Eller klona repot och bygg från källkod — vilket du redan har gjort.
4	torch.load() utan weights_only	Två filer i omnivoice/eval/ laddar checkpoints utan skyddet. Ingår inte i appens normala körväg — det är forskarverktyg för kvalitetsmätning.	Ignorera om du bara kör appen. Kör inte eval-verktygen mot nedladdade checkpoints du inte litar på.
5	LAN-delning	Funktionen binder en andra server till 0.0.0.0 — men bara när du aktivt slår på den, och den är PIN-skyddad.	Slå bara på den på nätverk du litar på. Exponera aldrig porten mot internet utan API-nyckel.
6	Beroendeaviseringar blockerar inte merge	pip-audit och bun audit rapporterar men stoppar inte en PR. Bara hemlighetsskanningen är blockerande.	Medveten avvägning från projektet. Uppdatera regelbundet — de rensar aviseringar i praktiken.

5. Sönt som talar starkt för projektet

- **Egen SSRF-testsvit.** Det finns tester som försöker lura appen att anropa 169.254.169.254 (molnleverantörers metadata-endpoint — klassisk nyckelstöld) och som

simulerar **DNS-rebinding**, där en värd först svarar 127.0.0.1 och sedan något annat. Det är inte nybörjarnivå.

- **Hemlighetsskanning blockerar merge.** gitleaks är hård grind i CI; CodeQL, bandit och beroendegranskning kör på varje PR och veckovis.
- **Genomtänkt behörighetsmodell** med separata nivåer — anonym, loopback, betrott nät, PIN, API-nyckel, adminsession — i stället för ett enda "är du inloggad"-flagga.
- **Redigering är genomgående.** Egna moduler för sanering av felmeddelanden, sökvägssäkerhet, CSRF och diagnostikbuntar. Loggar HF-token-redigeras innan de visas.
- **Publik säkerhetspolicy** med privat rapporteringskanal och en uttrycklig policy om modellförsörjningskedjan.
- **AGPL-3.0** — ingen "open source-tvätt" med kommersiella tilläggs klausuler.

6. Min rekommendation till dig

Trygg att köra — med tre enkla vanor

1. Kör från källkod (du har redan klonat repot som submodule), eller verifiera SHA256 om du hämtar en release.
2. Låt telemetrin vara avstängd — den är det som standard. Vill du vara helt säker, sätt `OMNIVOICE_ANALYTICS_DISABLED=1`.
3. Lägg aldrig till en modellfil från en privat källa, hur frestande erbjudandet än är. Det är den enda vägen in som projektet inte kan stänga åt dig.

En reservation, för hederlighetens skull: jag har granskat 2 240 filer med mönstersökning och riktad läsning av de säkerhetskritiska delarna — inte rad för rad genom hela kodbasen. En mycket välgömd bakdörr i en enskild motorintegration skulle kunna undgå den här metoden. Sannolikheten är låg med tanke på projektets öppenhet, CI-kedja och aktiva granskningsbottar, men den är inte noll.

Källor: lokal granskning av commit eaf8bb9 · CHANGELOG.md · .github/SECURITY.md · .github/workflows/security.yml · GitHub Security Advisories (tom) · projektets issue-historik